

Non-Empty Stdout Weak-Oracle Risks

This note separates two facts:

- In `strict_io_results_clean_full.json`, there are no observed non-empty stdout-equal false positives. The confirmed false positives in that rerun are all empty-stdout matches.
- The dataset still contains benchmarks whose stdout is non-empty but too weak to prove semantic correctness. These are plausible future false positives if a generated implementation prints the same summary while computing wrong values.

0.1. Clean Strict Rerun Finding

field	value
matched candidates	234
stdout false positives	156
empty-stdout false positives	156
non-empty stdout-equal false positives	0
non-empty stdout but stdout differs	4

Thus the current L40S clean rerun does not provide an evidenced example of:

```
candidate stdout != ""
candidate stdout == reference stdout
candidate implementation is semantically wrong
```

The best thing we can do from the present artifact is identify high-risk benchmarks where the printed output would be a weak oracle.

0.2. Best Benchmark-Design Risk: T/relu_batch_norm_conv2d

0.2.1. Why This Is The Right Subclass

Reference source:

`experiments/tritonbench_l40s_20260518_184231/source_data/TritonBench_T_v1/relu_batch_norm_conv2d.py`

The reference computes:

```
conv_result = F.conv2d(input, weight, bias=bias, stride=stride,
                        padding=padding, dilation=dilation, groups=groups)
bn_result = F.batch_norm(conv_result, running_mean, running_var,
                          bn_weight, bn_bias, training=training,
                          momentum=momentum, eps=eps)
return F.relu(bn_result, inplace=inplace)
```

The test prints only:

```
print(f"Output tensor shape: {output_tensor.shape}")
assert output_tensor.shape == (4, 6, 32, 32)
```

The tested inputs are:

input	value
input_tensor	<code>torch.randn(4, 3, 32, 32)</code>
weight_tensor	<code>torch.randn(6, 3, 3, 3)</code>
bias_tensor	<code>torch.randn(6)</code>
running_mean	<code>torch.zeros(6)</code>
running_var	<code>torch.ones(6)</code>
bn_weight	<code>torch.ones(6)</code>
bn_bias	<code>torch.zeros(6)</code>
stride/padding/dilation/groups	<code>1 / 1 / 1 / 1</code>
training	<code>True</code>

So a generated implementation could be semantically wrong and still pass this kind of stdout test if it returns a tensor with shape (4, 6, 32, 32) and prints the same line. However, after manually inspecting the best materialized candidates, none of the concrete candidates below are evidenced passes under the appended gold test tail: they are semantically wrong, but they likely fail before printing or fail to run.

0.2.2. Generated Candidate 1: GPT-4o

Candidate:

materialized_candidates/Bench_T_general_purpose/output_gpt-4o/def_0072.py

Representative generated code:

```
output = torch.empty((N, K, out_H, out_W), device=input.device)
grid = (out_H * out_W,)
relu_batch_norm_conv2d_kernel[grid](
    input, weight, bias,
    running_mean, running_var, bn_weight, bn_bias,
    output, stride, padding, dilation, groups,
    eps, inplace, H, W, C, K, stride_h, stride_w, block_size=128
)
return output
```

Inside the kernel, the computation is:

```
input_val = tl.load(input_ptr + h_idx * stride_h + w_idx * stride_w)
weight_val = tl.load(weight_ptr)
bias_val = tl.load(bias_ptr)
conv_result = input_val * weight_val + bias_val
normalized = (conv_result - mean) / tl.sqrt(var + eps) * bn_weight + bn_bias
tl.store(output_ptr + h_idx * W + w_idx, relu_result)
```

Why it is wrong:

- It does not perform a 3x3 convolution over all input channels.
- It reads only one weight value and one spatial input value.
- It stores only a single spatial plane, leaving most batch/output-channel entries uninitialized.
- It uses running statistics, while the tested reference calls `F.batch_norm(..., training=True)`, which uses batch statistics.

Would the weak stdout pass?

No, not as-is. The gold test tail constructs CPU tensors. This candidate immediately launches a Triton kernel on those CPU tensors, so it should fail before the gold tail reaches:

Output tensor shape: `torch.Size([4, 6, 32, 32])`

The important point is narrower: if the same implementation were made runnable on CUDA and returned its allocated output shape, the shape-only stdout would not detect the value errors below.

Counterexample:

```
torch.manual_seed(0)
input = torch.randn(4, 3, 32, 32, device="cuda")
weight = torch.randn(6, 3, 3, 3, device="cuda")
bias = torch.randn(6, device="cuda")
running_mean = torch.zeros(6, device="cuda")
running_var = torch.ones(6, device="cuda")
bn_weight = torch.ones(6, device="cuda")
bn_bias = torch.zeros(6, device="cuda")

gt = ref.relu_batch_norm_conv2d(
    input, weight, bias, padding=1,
    running_mean=running_mean, running_var=running_var,
    bn_weight=bn_weight, bn_bias=bn_bias, training=True,
)
bad = cand.relu_batch_norm_conv2d(
    input, weight, bias, padding=1,
```

```

        running_mean=running_mean, running_var=running_var,
        bn_weight=bn_weight, bn_bias=bn_bias, training=True,
    )

```

```

assert bad.shape == gt.shape
torch.testing.assert_close(bad, gt)

```

The shape assertion can pass; the value assertion should fail.

0.2.3. Generated Candidate 2: Qwen2.5-72B

Candidate:

materialized_candidates/Bench_T_general_purpose/output_Qwen2.5-72B-Instruct/def_0072.py

Representative generated code:

```

output_shape = (batch_size, out_channels, out_height, out_width)
output = torch.empty(output_shape, device=input.device, dtype=input.dtype)
grid = (batch_size * out_height * out_width // 1024, 1, 1)
conv2d_batch_norm_relu_kernel[grid](...)
return output

```

Why it is wrong:

- The launch grid can be zero for small spatial products.
- The kernel treats Python shape tuples as Triton values.
- Batch normalization is computed from the local accumulator, not PyTorch's training batch statistics over the convolution result.
- The convolution indexing does not implement the full PyTorch conv2d contract.

Would the weak stdout pass?

No, not as-is. Like the GPT-4o candidate, the gold test tail uses CPU tensors and this implementation launches Triton, so it should fail before the print.

The benchmark-design risk remains: this candidate computes `output_shape = (4, 6, 32, 32)` for the baked-in benchmark input, exactly the shape that the reference test prints and asserts. If it were made runnable, that stdout line would still not prove semantic correctness.

Counterexample:

```

assert cand.relu_batch_norm_conv2d(...).shape == (4, 6, 32, 32)
torch.testing.assert_close(cand.relu_batch_norm_conv2d(...),
                           ref.relu_batch_norm_conv2d(...))

```

The first line can pass while the second fails.

0.2.4. Generated Candidate 3: Qwen Coder-Specific

Candidate:

materialized_candidates/Bench_T_coder_specific/qwen/def_0072.py

Representative generated code:

```

output_shape = (input_shape[0], weight_shape[0],
                input_shape[2], input_shape[3])
output = input.new_empty(output_shape)
relu_batch_norm_conv2d_kernel[grid=blocks_per_grid, block=threads_per_block](...)
return output

```

Why it is wrong:

- The output shape assumes stride 1 and padding preserving height/width.
- The kernel uses invalid scalar indexing and shape objects inside Triton JIT.
- The batch-norm computation is not equivalent to `F.batch_norm` with `training=True`.
- The grouped/dilated convolution contract is not implemented.

Would the weak stdout pass?

No. This candidate is syntactically invalid Python because the kernel launch uses keyword syntax inside the subscript:

```
relu_batch_norm_conv2d_kernel[
    grid=blocks_per_grid,
    block=threads_per_block,
    num_warps=4
](...)
```

So it cannot import, the gold tail cannot run, and no stdout is printed. If that syntax were repaired, it still hard-codes an output shape that happens to match the baked-in padding=1, stride=1 case while not implementing the real operator.

0.3. Other Non-Empty Stdout Risk Patterns

These were identified by static inspection, but no materialized non-empty stdout-equal false positive was evidenced in the clean strict result file.

case	stdout	risk
G/diag_ssm_triton	prints tensor summaries	large tensor repr can hide interior values
G/int8_matmul_kernel	prints result_gold dict	large tensor repr may truncate values
G/log_softmax	prints result dict	only fixed shapes and dim=-1 are covered
G/matrix_transpose	prints result dict	limited baked-in shapes
T/matrix_multiply_symmetric	reference docstring has a print example, executable test tail does not print	direct candidates exist, but gold stdout is empty

0.4. Conclusion

For this artifact, the non-empty stdout-equal false-positive bucket is empty. The strongest benchmark-design risk is relu_batch_norm_conv2d: the reference test prints and asserts only output shape. But the concrete materialized candidates inspected here are not evidenced stdout-equal passes; they appear to fail before printing, or fail to import. The result is a useful warning about shape-only stdout tests, not a confirmed non-empty stdout false positive.